

can you define the perfect folder structure for a web app that has a fastapi backend , react frontend with nginx, sqlmodel, docker containers for components, alembic etc. all with python?

A clean structure for a production-grade app should optimize for:

- clear separation of concerns
- scalability
- local development simplicity
- containerization
- testability
- CI/CD friendliness
- future microservice extraction if needed

Here's a strong modern structure for:

- FastAPI backend
- React frontend
- NGINX reverse proxy
- SQLAlchemy + Alembic
- Docker / Docker Compose
- Python tooling
- async-ready architecture
- production deployment readiness

Recommended Project Structure

```
myapp/  
|  
├── README.md  
├── .gitignore  
├── .env  
├── .env.example  
├── docker-compose.yml  
├── docker-compose.dev.yml  
└── Makefile
```

```
├─ pyproject.toml
├─ scripts/
│   ├── dev.sh
│   ├── prod.sh
│   ├── migrate.sh
│   └─ seed.sh
│
├─ infra/
│   ├── nginx/
│   │   ├── nginx.conf
│   │   └─ conf.d/
│   │       └─ default.conf
│   │
│   ├── postgres/
│   │   └─ init.sql
│   │
│   └─ docker/
│       ├── backend.Dockerfile
│       ├── frontend.Dockerfile
│       └─ nginx.Dockerfile
│
├─ backend/
│   ├── alembic.ini
│   ├── pyproject.toml
│   ├── requirements/
│   │   ├── base.txt
│   │   ├── dev.txt
│   │   └─ prod.txt
│   │
│   ├── alembic/
│   │   ├── env.py
│   │   ├── script.py.mako
│   │   └─ versions/
│   │
│   ├── app/
│   │   ├── main.py
│   │   ├── config/
│   │   │   ├── settings.py
│   │   │   ├── logging.py
│   │   │   └─ security.py
│   │   │
│   │   ├── api/
│   │   │   ├── deps.py
│   │   │   ├── router.py
│   │   │   └─ v1/
│   │   │       ├── auth.py
│   │   │       ├── users.py
│   │   │       └─ health.py
│   │   │
│   │   └─
│   │
│   └─
│
└─
```

```
| | | └─ core/
| | |   └─ db.py
| | |   └─ lifespan.py
| | |   └─ middleware.py
| | |   └─ exceptions.py
| | |   └─ events.py
| | |
| | | └─ models/
| | |   └─ user.py
| | |   └─ base.py
| | |
| | | └─ schemas/
| | |   └─ user.py
| | |   └─ auth.py
| | |   └─ common.py
| | |
| | | └─ repositories/
| | |   └─ base.py
| | |   └─ user.py
| | |
| | | └─ services/
| | |   └─ auth_service.py
| | |   └─ user_service.py
| | |
| | | └─ utils/
| | |   └─ jwt.py
| | |   └─ hashing.py
| | |   └─ validators.py
| | |
| | | └─ tests/
| | |   └─ conftest.py
| | |   └─ test_auth.py
| | |   └─ test_users.py
| | |
| | | └─ workers/
| | |   └─ celery_app.py
| | |   └─ tasks.py
| | |
| | └─ Dockerfile
|
└─ frontend/
  └─ package.json
  └─ vite.config.ts
  └─ tsconfig.json
  └─ public/
  |
  └─ src/
    └─ main.tsx
    └─ App.tsx
```

```

├── api/
│   ├── client.ts
│   └── auth.ts
├── assets/
├── components/
│   ├── common/
│   └── layout/
├── features/
│   ├── auth/
│   └── users/
├── hooks/
├── pages/
├── routes/
├── store/
├── styles/
├── types/
└── utils/
├── tests/
└── Dockerfile
├── nginx/
│   ├── nginx.conf
│   └── conf.d/
│       └── app.conf
└── docs/
    ├── architecture.md
    ├── api.md
    ├── deployment.md
    └── decisions/

```

Why This Structure Works

1. Backend is Domain-Oriented

Instead of giant monolithic files:

```
models/  
services/  
repositories/  
api/
```

This keeps:

- business logic separate
- DB logic separate
- transport layer separate
- validation separate

This scales extremely well.

Recommended FastAPI Architecture

Use:

```
Route -> Service -> Repository -> DB
```

Example flow:

```
/users endpoint  
↓  
UserService  
↓  
UserRepository  
↓  
SQLModel session
```

This prevents your routes from becoming giant messes.

SQLModel + Alembic Layout

Best practice:

```
models/  
schemas/
```

Keep:

- SQLAlchemy tables in `models`
- API request/response models in `schemas`

Even though SQLAlchemy mixes concepts, separating API schemas still helps long-term.

Recommended DB Setup

```
# core/db.py  
  
from sqlalchemy import Session, create_engine  
  
engine = create_engine(settings.DATABASE_URL)  
  
def get_session():  
    with Session(engine) as session:  
        yield session
```

Alembic Best Practice

Keep migrations isolated:

```
alembic/  
versions/
```

Never auto-generate blindly in production.

Use:

```
Bash
```

Then manually inspect.

Frontend Structure

The important part:

```
features/
```

Feature-based architecture scales FAR better than:

```
components/  
pages/  
utils/
```

alone.

Example:

```
features/  
  auth/  
    components/  
    hooks/  
    api/  
    types/
```

This prevents “global soup.”

NGINX Placement

Keep infra concerns outside frontend/backend:

```
infra/nginx/
```

Good for:

- Kubernetes later
- Terraform later
- ECS later
- CI/CD later

Docker Philosophy

Each component gets its own container:

- backend
- frontend
- nginx
- postgres
- redis
- worker

Example:

```
services:
  backend:
  frontend:
  nginx:
  postgres:
  redis:
```

This mirrors real production infrastructure.

Recommended Docker Compose

```
services:
  backend:
    build:
      context: .
      dockerfile: infra/docker/backend.Dockerfile

  frontend:
    build:
      context: .
      dockerfile: infra/docker/frontend.Dockerfile

  nginx:
    build:
      context: .
      dockerfile: infra/docker/nginx.Dockerfile
```


Recommended Backend Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY backend/requirements/prod.txt .

RUN pip install --no-cache-dir -r prod.txt

COPY backend .

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0"]
```

Recommended Frontend Dockerfile

Multi-stage build:

```
FROM node:20 AS build

WORKDIR /app

COPY frontend/package*.json ./

RUN npm install

COPY frontend .

RUN npm run build

FROM nginx:alpine

COPY --from=build /app/dist /usr/share/nginx/html
```

Recommended NGINX Reverse Proxy

```
server {  
    listen 80;  
  
    location /api {  
        proxy_pass http://backend:8000;  
    }  
  
    location / {  
        proxy_pass http://frontend:80;  
    }  
}
```

Recommended Tooling

Backend

Use:

- FastAPI
- SQLAlchemy
- Alembic
- Pydantic Settings
- Ruff
- Pytest
- Uvicorn
- httpx

Optional:

- Celery
- Redis
- APScheduler

Frontend

Use:

- React
- TypeScript
- Vite

- TanStack Query
- React Router
- Zustand or Redux Toolkit

Recommended Environment Management

Use:

```
.env
.env.example
```

Never commit secrets.

Example:

```
DATABASE_URL=postgresql+psycopg://user:pass@postgres/db

env
```

Recommended Dev Commands

Use a Makefile:

```
dev:
    docker compose up --build

migrate:
    docker compose exec backend alembic upgrade head

makemigrations:
    docker compose exec backend alembic revision --autogenerate -m "$(m)"

test:
    docker compose exec backend pytest
```

The Biggest Mistake to Avoid

Avoid:

```
routes/  
crud/  
models/
```

with all business logic inside routes.

That architecture collapses at scale.

Use:

```
api/  
services/  
repositories/
```

instead.

If You Want the “Ideal” Modern Stack

This is what many senior teams now use:

Backend

- FastAPI
- SQLAlchemy
- PostgreSQL
- Alembic
- Redis
- Celery
- Pydantic Settings
- Ruff
- Pytest

Frontend

- React
- TypeScript
- Vite
- TanStack Query

- Zustand
- Tailwind

Infra

- Docker
- NGINX
- GitHub Actions
- Traefik or Caddy later
- Kubernetes later if needed

Recommended Future-Proof Enhancements

Add later:

```
backend/app/domain/  
backend/app/events/  
backend/app/tasks/  
backend/app/cache/
```

when scaling.

Final Recommendation

The most important architectural decisions are:

1. Feature/domain separation
2. Service layer
3. Repository layer
4. Container isolation
5. Clear infra separation
6. Environment consistency
7. Migration discipline
8. Type-safe frontend/backend contracts

Those matter much more than tiny folder naming choices.



